Analysis of a Božena Němcová's Literary Technique of Ending Sentences with Softer Words: Final Report

Vojtěch Kubala

Computer Science, University of Canterbury

DATA301 - Big Data Computing and Systems

Dr. Francesco Maiorana

June 1, 2026

Summary

The Grandmother (in the original *Babička*) is a Czech novella written in 1854 by Božena Němcová and is considered highly significant in Czech culture. I examine the following research question: Does Božena Němcová, in her book *The Grandmother* deliberately use softer words at the ends of sentences? I use a dataset consisting of the book *The Grandmother*, 50 other prominent Czech literary pieces, Czech Wikipedia articles, and the results of a Qualtrics survey. I analyse them using pipelining, mapping, filtering, array multiplication, grouping, sampling, survey administration, divide-and-conquer through parallelism, shingling, relative difference, and t-tests. The difference between the softness of the last words and the second-to-last words was significantly more positive in *The Grandmother* when compared to other books, but not when compared to the *Czech Wikipedia*. The difference between the softness of the last words and the rest of the sentence was significantly more positive in *The Grandmother* when compared both to other books and to the *Czech Wikipedia*. The results provide partial evidence that Němcová deliberately used softer words to end sentences.

Introduction

Background:

The Grandmother (in the original *Babička,* 2012) is a Czech novella written in 1854 by Božena Němcová. The book is considered significant in the Czech culture and is one of the few widely recognized Czech literary works written by a woman. It describes a grandmother's life in the countryside and includes references to historical events. In one literary analysis, Jan Mukařovský asserts that Němcová systematically orders last words in a sentence so that the sentence-ending words are lower in tone and have weaker accentuation (Nebrozanský, n.d.).

The dataset contains the first 51 books written by Czech authors as featured on *PDF KNIHY.* Each book has a name, author, and text. In addition, it contains the latest Wikipedia pages and articles dataset (as of 30. April 2026) (*Wikimedia Foundation*, 2026). The results of an online survey on perceived softness of consonant phonemes are also included and contain the following features: 1) *Finished:* "True" or "False" indicating whether the survey was completed, 2) *Distribution Channel:* name of the survey distribution method, 3) *D1:* 'True' or 'False' indicating whether the participant was a native Czech speaker, 4) *A1_1 – A28_1:* Softness ratings of phonemes on a scale from 0 to 100.

Several algorithmic techniques were used for data processing and analysis. 1) *Pipelining:* The sequential application of functions where each function takes the output of the previous function as its input. 2) *Array multiplication:* The dot product of two 1-dimensional arrays is taken to calculate a word's softness. 3) *Divide and Conquer through Parallelism:* Using *Dask,* the data is split into partitions which are then processed in parallel. The partition results are then reduced to values representative of the whole dataset. 4) *Shingling:* text was split into two-character combinations contained within each word. 5) *Relative Difference:* The difference between the normalised dictionaries obtained through shingling, used to examine which shingles are characteristic of last words compared to second-to-last words. 6) *T-tests:* "A ratio that quantifies how significant the difference is between the 'means' of 2 groups while considering their variance or distribution" (Wadhwa & Marappa-Ganeshan, 2023).

Motivation

I recently listened to the audio version of *The Grandmother*. I did so with the intention to reconnecting with my Czech roots and to learn more about Czech culture. I could not help but notice how soothing the sound of the text was and how beautifully and skilfully

the flexibility of the Czech language was exploited. So far, the analyses of the text have been of mostly qualitative nature, and obtaining quantitative evidence could help confirm some of these observations. This research could also help me share about my Czech culture and my interest in the Czech language.

Research Question

My research question is: Does Božena Němcová, in her book *The Grandmother,* deliberately use softer words at the ends of sentences? Each component of the dataset meaningfully contributes to answering this question: 1) The book *The Grandmother* itself is the subject of analysis, 2) the 50 other books and the Czech Wikipedia provide a baseline against which *The Grandmother* is compared, and 3) the survey dataset determines how softness is calculated for each word. The quantitative analysis provides insights into the last words as compared to other words from the same book and to last words from other bodies of text, and it partially validates observations derived from qualitative analyses.

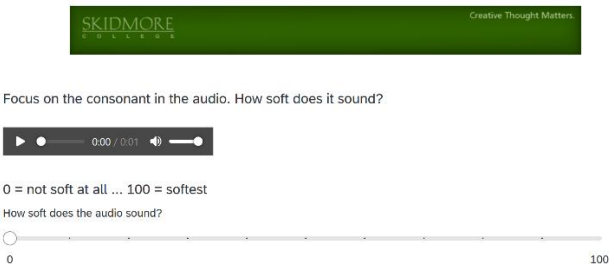
Experimental Design and Methods

Operationalization of *Softness* and Survey

Perception of word softness is influenced by many unknown variables and is inherently subjective. To ensure that my measure does not rely solely on my discretion, I created an online Qualtrics survey, in which users were asked to rate an audio recording of each phoneme for softness. The list of phonemes was obtained from *Transkripce češtiny Pomocí IPA*. The order of questions was randomized.

Figure 1

A sample question from a survey evaluating perception of softness



To measure softness quantitatively, I operationalize softness as the mean softness of each phoneme in a word, where vowels are assumed to have maximal softness and consonant phoneme softness is defined as their mean rating from the online survey.

Algorithm and Data Flow

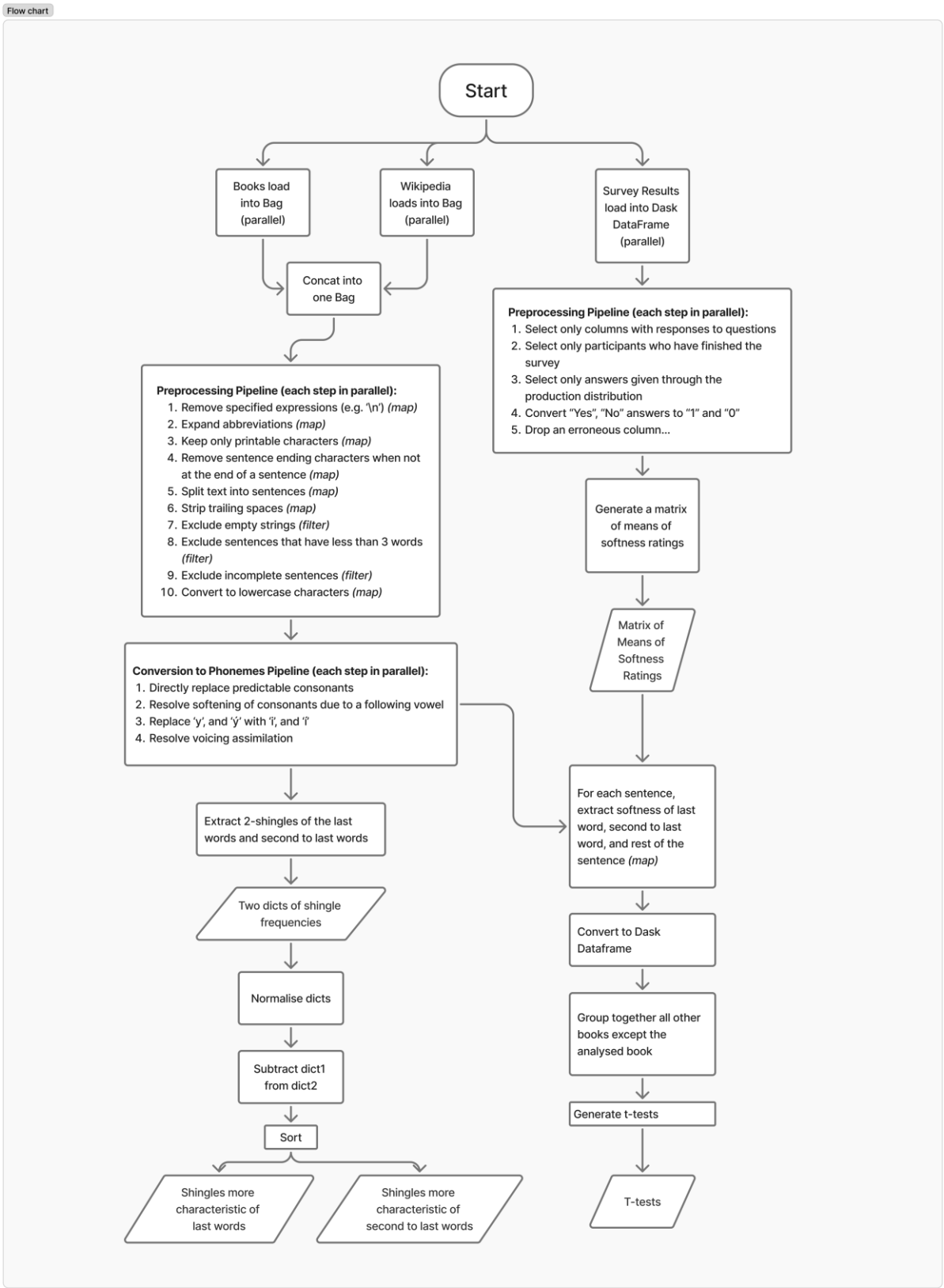
All the books and Wikipedia articles are loaded in parallel into Dask Bags and concatenated together. All the text then goes through a preprocessing pipeline that uses functions `map` and `filter` to convert the text to complete lowercase sentences with only letter characters. The sentences are then converted to phonemes using rules outlined in Ústav pro jazyk český AV ČR (2008–2026). In parallel with the previous steps, the survey results are loaded into a Dask DataFrame and pre-processed to contain only relevant columns in a numeric format. The mean softness of each phoneme is then calculated and added to a NumPy array. Using this array, three metrics are produced for each sentence in the Bag: 1) softness of the last word, 2) softness of the second-to-last word, 3) softness of the rest of the sentence (excluding the last word). The measures are converted to a Dask DataFrame and the relative differences between values are added. All other books (except the analysed one) are

grouped together in a category called “other”. T-tests are then generated using the Scipy library (Virtanen et al., 2020) to examine the difference between the last words and other parts of the sentence. T-tests are also used to evaluate the differences between groups in the softness differences.

To further explore what characterizes the last and second-to-last words, shingles representing each of these positions are extracted from each sentence as dictionaries of frequencies. The dictionaries within each group are joined with the values of matching keys summed. The dictionaries are normalized and subtracted from each other. The resulting dictionary is sorted by value and the first 10 and last 10 values are output to represent the shingles most characteristic of the last words and the second-to-last words respectively.

Figure 2

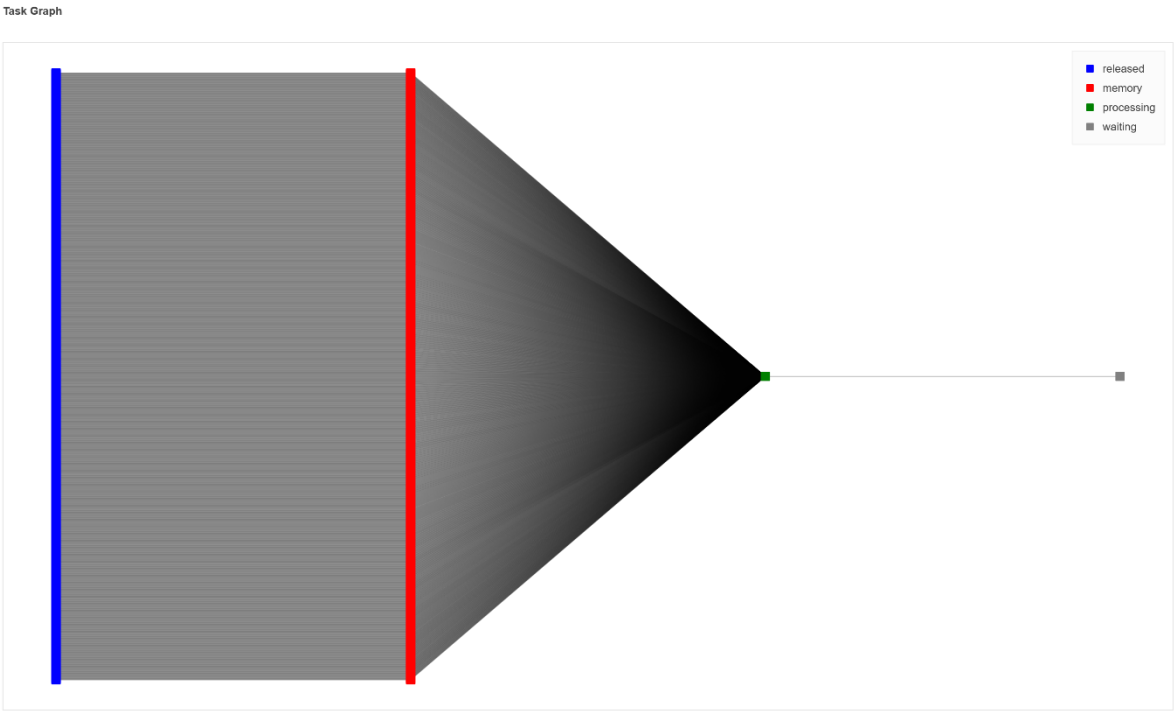
A high-level outline of the analysis algorithm



Initially, there were several performance issues. I used `dask.distributed` to run a local cluster which allowed me to examine the graph of tasks and identify that one of the problems was that Dask partitioned Wikipedia into more than 1300 small partitions.

Figure 3

A task graph of the unoptimized algorithm



I followed Dask's recommendation and used the `repartition` function which by itself reduced the preprocessing time substantially. I also added the possibility of randomly sampling Wikipedia for debugging and grading purposes. Furthermore, I encountered an issue with `pdfium` when directly converting the Dask Bag into a Dask DataFrame. I resolved the problem by persisting all measures before the conversion.

List of Libraries and Functions Used

In my project I used the following libraries:

1. Dask – parallel computing, and performance monitoring
2. Pandas – non-parallel but flexible processing of DataFrames
3. Numpy – efficient array multiplication
4. PyPdfium2 – fast extraction of text from PDF files
5. Re – regular expressions
6. Math – square root function
7. Scipy – functions for performing t-tests
8. Matplotlib – creation of performance plots

I also created several custom functions. The following are the most significant functions I wrote for text preprocessing and analysis:

1. book_loader: loads books and converts them from PDF to a string
2. generate_wiki_urls: generates a list of urls of all the Wikipedia files
3. expand_abbr: expands most common abbreviations to their full form
4. remove_non_ending_stops: removes sentence ending characters outside of the context of a sentence end
5. voice_group: takes a group of consonants from a word as an argument and modifies their voicing according to appropriate rules and exceptions
6. match_voice_paired: finds consonant groups and ensures all of them are correctly voiced
7. match_voice_son: voices out parts of sentences affected by sonorant consonants
8. voiceless_rr: handles the conversion of ř to r̥ in appropriate cases
9. voicing_assimilation: executes the process of voicing assimilation

10. expand_soft: expands syllables where either ‘ĕ’ or ‘i’ softens the previous consonant
11. to_phonemes: sentence to phonetic transcription
12. generate_matrix_of_means: loads results of the survey into a NumPy array
13. word_array: converts text to a NumPy array that represents the frequency of each consonant phoneme
14. sentence_to_softness: converts a sentence to a tuple with a source name, softness of the last word, softness of the second-to-last word, and softness of the rest of the sentence
15. gen_t_tests: generates t-tests to examine the significance between two columns of data for the same group
16. gen_between_groups_t_tests: generates t-tests to examine the significance of relative measures between groups
17. s_pooled: calculates the pooled standard deviation of two groups of data using the method described by (Thalheimer & Cook, 2002)
18. word_to_2shingles: outputs a dictionary representing the frequency of each 2-shingle represented in the word
19. add_dicts: performs a join on two dictionaries while summing values at overlapping keys
20. normalise_dict: normalises the dictionaries so that each value represents the relative frequency of the tuple
21. measure_perf: measures the performance of the entire analysis pipeline with a specified sample size of Wikipedia and books

Results

Paired samples t-tests suggest the following: For the book *The Grandmother,* there is no significant difference between the softness of the last word and that of the second-to-last word $t(4282) = -0.79, p = .43$, and between the softness of the last word and the rest of the sentence $t(4282) = -1.18, p = .24$. For the other books, the second-to-last word was significantly softer than the last word $t(169130) = -21.85, p < .001$, and the rest of the sentence was significantly softer than the last word $t(169130) = -17.53, p < .001$. For the Czech Wikipedia, the second-to-last words were significantly softer than the last words $t(11014428) = -86.87, p < .001$, and the rest of the sentence was significantly softer than the last word $t(11014428) = -248.70, p < .001$. The independent-samples t-tests suggest the following: The value (softness of the last word – softness of the second-to-last word) was significantly larger in the book *The Grandmother,* than in the group of other books, $t = 3.76, p < .001$, but there was no significant difference between *The Grandmother* and Wikipedia, $t = 1.38, p = .17$. 2) The value (softness of the last word – the softness of the rest of the sentence) was significantly larger between *The Grandmother* and both the rest of the books, $t = 2.32, p = .02$, and Wikipedia, $t = 5.54, p < .001$. *The* exploratory analysis identified the following shingles as those most characteristic of last words: “ku”, “ro”, “ce”, “ra”, “ki”, “ka”, “st”, “in”, “oc”, “or” and the following shingles as most characteristic of second-to-last words: “ov”, “ck”, “éh”, “kí”, “ím”, “íx”, “ho”, “sk”, “ké”, “ňí”.

The results partially support a positive answer to the research question. In all other analysed bodies of text, the second-to-last word and the rest of the sentence were significantly softer than the last word. This pattern is not observed in *The Grandmother,* possibly due to the lack of statistical power. When comparing the differences between the softness of the last word and the second-to-last word and the last word and the rest of the

sentence using other books and Wikipedia as baseline, *The Grandmother* is significantly more positive in almost all cases. This supports the answer that there is a tendency for softer words to be used at the end of the sentences in *The Grandmother.* The only exception is the difference between the softness of the last and second-to-last words, where the difference between *The Grandmother* and Wikipedia was not significant. To identify the cause of this discrepancy, further analysis of general text patterns is needed.

The exploratory analysis provides a significant insight into the characteristics that distinguish the last and second-to-last words. Shingles characteristic of second-to-last word were softer than the shingles characteristic of last word. This is consistent with the pattern that second-to-last words are softer than last words. Furthermore, several shingles that are characteristic of the second-to-last word are common Czech adjective suffixes which raises the question of whether certain word categories, such as adjectives or prepositions tend to skew the results.

Performance Analysis

The performance of the entire analysis pipeline on a single CPU has been analysed. The algorithm was run in Google Colab on a CPU with the “High-RAM” setting enabled. The algorithm was run with ten different sample sizes, and the duration of each one was measured. The results are shown in Table 1.

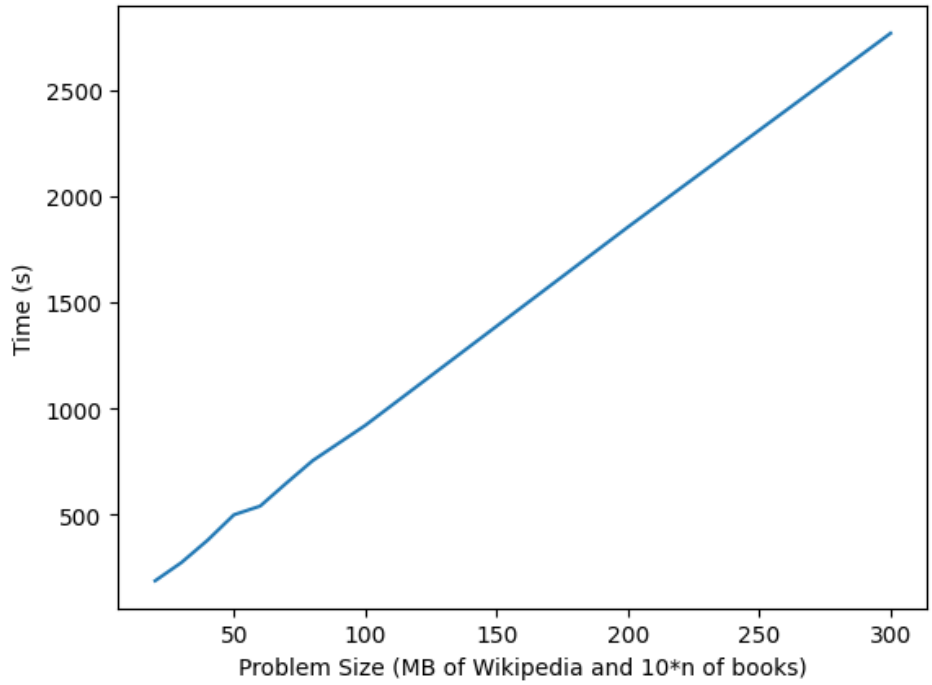
Table 1

Single machine performance measures

Wiki (MB)	20	30	40	50	60	70	80	100	200	300
Books	2	3	4	5	6	7	8	10	20	30
Time (s)	189	275	381	500	541	650	755	921	1855	2768

Figure 4:

Performance on a single machine



As evident from the graph, the time taken to complete the algorithm grows linearly with the problem size.

Weak scalability has also been assessed. The program utilizes `dask.distributed` to run on multiple CPUs with multiple processes. Only Wikipedia was included in the

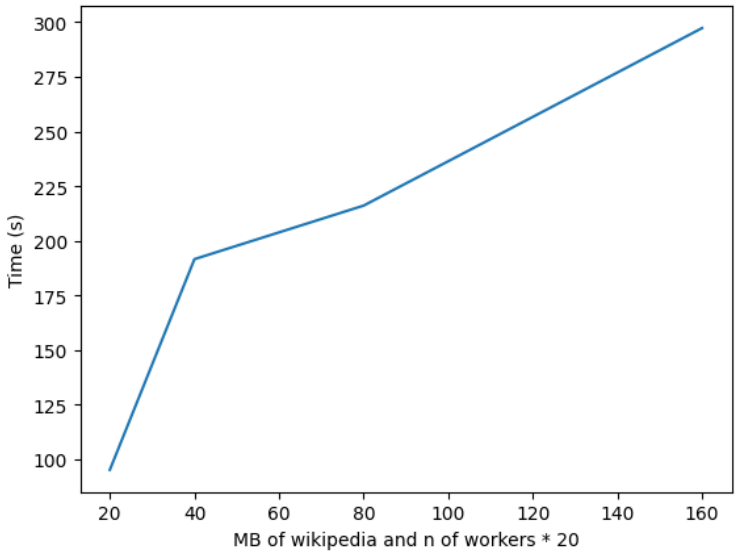
analysis because `pdfium` can cause problems during distributed computing. Four problem sizes have been tested on Google Cloud's n2-highmem-8 machine (Google, n.d.) with a proportionally increasing number of workers (8GB memory limit per worker). The results are shown in Table 2.

Table 2:

Wiki (MB)	20	40	80	160
Time (s)	95	192	216	297
Workers	1	2	4	8
$\mathbf{E_{weak}}$	1	0.49	0.43	0.32

Figure 5:

The graph of weak scalability



The algorithm scales moderately well as the number of workers increases. At eight workers, the overhead dominates the computation time. One contributing reason could be the limited resources of the machine used which caused some workers to fail in the last case.

Conclusion

The research question can be answered only partially. Most of the evidence suggests that Němcová deliberately used softer words at the ends of sentences, but there was no significant difference found between the softness of last words and second-to-last words when comparing *The Grandmother* with Wikipedia. The cause of this discrepancy must be examined further by identifying general patterns of softness in Czech sentences and by finding outliers that can skew the results. Examining Němcová's general style of writing on more pieces of literature could provide additional perspectives and statistical power.

The results imply that there are patterns to how softness is distributed in sentences and that last words in a sentence differ in softness from the rest of the words. The results provide some quantitative evidence that support previous qualitatively derived observation about Němcová's authorial choices. The analyses of literary pieces have largely been conducted from a subjective perspective, but the current method implies that it is possible to conduct precise quantitative analysis of literature. The performance of the algorithm allows for relatively large bodies of text to be analysed at scale and compared to other bodies of text.

The project could be continued by including more statistical methods to identify trends in word softness within a sentence and to remove outliers. Furthermore, the algorithm could also be applied on more books and texts to examine whether there are more pieces of literature that use softer words at the ends of sentences. Further analysis of work by Božena Němcová could also provide deeper and more generalizable information about her authorial choices.

Critique of Design and Project

There is one part that creates a significant bottleneck in the computation. It occurs when the text is loaded in the Bag, then separated into sentences, and then flattened. When analysing the entire Wikipedia, this results in more than one million objects being loaded into Dask. It creates significant overhead for the Dask scheduler to orchestrate the computation of so many objects. The overhead can outweigh the benefits of parallelism at large sample sizes. The following approach could be more effective:

1. All the text is loaded in parallel into the Bag
2. During preprocessing, the text is split into sentences but not flattened. This creates a Bag where each item is a list of tuples representing each sentence and its corresponding metrics.
3. The multiple sentences in each item would be processed serially.
4. The metrics would be aggregated serially within each item.
5. The metrics of all items would be reduced in parallel.

This approach would allow for the creation of fewer larger objects rather than many small objects and would thus reduce overhead for orchestrating the scheduling in Dask.

Reflection

Throughout the project, I used the following course concepts and tools: Dask, techniques for working with Dask data structures, MapReduce, time library, Google-Cloud-based performance analysis, pipelining, mapping, filtering, array multiplication, grouping, divide-and-conquer through parallelism, shingling, relative difference, scalability, weak scalability, and Gustafson's law. I also learned many new things during the project, some of which include techniques of natural language processing, regular

expressions, phonological transcription of the Czech language, extraction of text from PDF, downloading and analysing the entire Czech Wikipedia, pooling standard deviation, applying appropriate t-tests using `scipy`, using `dask.distributed`, analysing Dask performance using its dashboard, random sampling, and diagnosing performance using `dask.diagnostics`.

References

Link to Project Repository:

https://github.com/vk4444/DATA301_Project/tree/main

Students I have directly worked with on this project:

The project was completed individually in its entirety.

Works Cited

Attardi, G. (n.d.). *Wikiextractor/readme.md at master · Attardi/Wikiextractor*. GitHub. https://github.com/attardi/wikiextractor/blob/master/README.md

Dask. (n.d.). https://www.dask.org/

Google. (n.d.). *General-purpose machine family for Compute engine | google cloud documentation*. Google. https://docs.cloud.google.com/compute/docs/general-purpose-machines#n2-high-mem

Kumar, A. (2025, July 21). *I tested 7 python PDF extractors so you don't have to (2025 edition)*. Medium. https://onlyoneaman.medium.com/i-tested-7-python-pdf-extractors-so-you-dont-have-to-2025-edition-c88013922257

Nebrozanský. (n.d.). *Mukařovský - Pokus O Slohový Rozbor Babičky boženy němcové*. Nebrozanský web. https://gummidek.estranky.cz/clanky/mukarovsky---pokus-o-slohovy-rozbor-babicky-bozeny-nemcove.html

Němcová, B. (2012). Babička. Městská knihovna v Praze. http://web2.mlp.cz/koweb/00/03/37/76/44/babicka.pdf

PDF KNIHY. PDF knihy. (n.d.). https://pdfknihy.maxzone.eu/index.html

Thalheimer, W., & Cook, S. (2002). How to calculate effect sizes from published research: A simplified methodology. *Work-Learning Research*, *1*(9), 1-9.

Transkripce češtiny Pomocí IPA. Fonetický ústav. (n.d.). https://fonetika.ff.cuni.cz/o-fonetice/foneticka-transkripce/transkripce-cj-ipa/

Ústav pro jazyk český AV ČR. (2008–2026). *Internetová jazyková příručka* [online]. Praha: Ústav pro jazyk český AV ČR, v. v. i. https://prirucka.ujc.cas.cz/

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., Van Der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … Vázquez-Baeza, Y. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Wadhwa, R. R., & Marappa-Ganeshan, R. (2023). *T test*. In *StatPearls*. StatPearls Publishing. https://www.ncbi.nlm.nih.gov/books/NBK553048/

Wikimedia Foundation. (2026, March 24). *Wikipedie:Stáhnutí Databáze*. Wikipedia. https://cs.wikipedia.org/wiki/Wikipedie:St%C3%A1hnut%C3%AD_datab%C3%A1ze

AI Acknowledgements

LLMs were used rarely throughout this project and never to generate larger blocks of text or code.

Microsoft Copilot was used to write down the following list of consonant phonemes and vowels: consonant_phonemes = ["p","b","t","d","ťˇ","ďˇ","k","f","v","s","z","š","ž","c","č","ʣ","ʤ","m","n","ň","j","r","l","ř","ɬ̊","h","x"]

vowels = ['a', 'á', 'e', 'é', 'i', 'í', 'o', 'ó', 'u', 'ú', 'ů', 'y', 'ý']

The following conversation helped explain a performance bottleneck in the project:

Search Engine: Github Copilot

Date and Time: 25/05/2026 20:48

Search query: why is the flatten operation being so expensive when converting used on a dask bag of lists of sentences. I would like to analyze and parallelyze the text at sentence level, but whenever I try to do that, the performance is severely hindered
Output: Optimized tool selection

Starting: *Suggest code improvements for efficiency* (3/4)

`flatten()` is expensive in your pipeline because it's the moment where you turn "a few large items" into "millions of tiny Python objects", and Dask Bag (and Python) pay a lot of overhead per element.

In code(1).ipynb.ipynb) you effectively do:

- `to_sentences()` returns a **Python list** of sentence strings (allocates lots of objects at once).

- `.flatten()` then **concatenates/iterates those lists** to create a new bag whose *logical* element count explodes.

- Every later `.map/.filter` now runs **per sentence**, so you’re doing millions of Python calls + regex/split work + (with `distributed`) serialization between worker tasks.

Why it hurts so much

- **Python object overhead dominates**: a 1–2 MB text chunk can become tens of thousands of sentence strings; the bytes aren’t the problem, the *object count* is.

- **Serialization/network overhead (distributed)**: millions of small strings moving between tasks/workers is far slower than keeping work inside one task and emitting only aggregated results.

- **Low partition parallelism makes it worse**: with `files_per_partition=50` you create very few partitions, so each task produces a gigantic list-of-sentences, then `flatten` has to churn through it.

Best fix: avoid `flatten()` by emitting sentences directly at partition level

Instead of “map to list, then flatten”, do one `map_partitions` that *yields* sentences:

```python
import re

split_re = re.compile(r"[.!?:]")

def partition_to_sentences(partition):
    for name, text in partition:
        for s in split_re.split(text):
            s = s.strip()
            if s:
                yield (name, s)
```

sentences = all_text.map_partitions(partition_to_sentences)

```

Even better: push your cleaning + filters into that same generator so each sentence is processed once (no extra passes).

**If you truly need sentence-level data**

- Keep partitions reasonably sized (e.g., `files_per_partition=1..5`, not 50), so sentence work is spread across cores.

- Avoid calling `.compute()` on the full sentence bag; prefer computing **aggregations** (counts, frequencies, summary stats) or `persist()` and continue downstream.

If you tell me roughly how many sentences you end up with (e.g., `sentences.count().compute()`), I can recommend a good target for `files_per_partition` / `npartitions` so you get parallelism without drowning in per-sentence overhead.

Completed: *Suggest code improvements for efficiency* (3/4)
```

Searches Documentation

Search Engine: Google

Date and Time: 07/05/2026 15:46

Search query: public psychology datasets

Link visited with pros and cons for each link visited:

https://guides.library.ucla.edu/psychology/data: Lists only services, requires more searching within them

https://osf.io/th8ew/overview: Contains a link to a spreadsheet with multiple links at https://docs.google.com/spreadsheets/d/1ejOJTNTL5ApCuGTUciV0REEEAqvhI2Rd2FCoj7afops/edit#gid=0. This spreadsheet is very useful since it also contains the number of subjects in each dataset.

Link chosen: https://osf.io/th8ew/overview

Search Engine: Google

Date and Time: 07/05/2026 15:46

Search query: české knihy dataset

Link visited with pros and cons for each link visited: none

Link chosen: none

Search Engine: Google

Date and Time: 07/05/2026 15:46

Search query: české knihy dataset txt

Link visited with pros and cons for each link visited: https://www.nulk.cz/ek-obsah/cgi-bin/ceskylid/database/database.txt directly contains dataset without any description or documentation. Not very helpful.

https://www.ceska-poezie.cz/data-modely-literatura contains data that were manually annotated by experts about poetry. Very helpful and very data-analysis-ready.

Link chosen: https://www.ceska-poezie.cz/data-modely-literatura

---

Search Engine: Google

Date and Time: 07/05/2026 15:46

Search query: babička kniha

Link visited with pros and cons for each link visited: https://cs.wikipedia.org/wiki/Babi%C4%8Dka_(kniha) very interesting, provides context and insight into the book babička.

Link chosen: https://cs.wikipedia.org/wiki/Babi%C4%8Dka_(kniha)

Search Engine: Google

Date and Time: 07/05/2026 15:46

Search query: Mukařovský (1982), s. 691. babička

Link visited with pros and cons for each link visited: https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://edicee.ucl.cas.cz/data/sborniky/kongres/tretiIII/13.pdf&ved=2ahUKEwjk2aGLvYyUAxUBSWwGHXZkGeQQFnoECB0QAQ&usg=AOvVaw1HCZs-nMmACEyPjyPgi-ju, not very helpful since it is not by the author I needed

Link chosen: **none**

Search Engine: Google

Date and Time: 07/05/2026 15:46

Search query: Mukařovský (1982)

Link visited with pros and cons for each link visited: https://muj-antikvariat.cz/kniha/studie-z-poetiky-mukarovsky-jan-1982?__cf_chl_tk=bVS0FGMl51Y6DzdOW9fDcNCl_dH4DEm5wq.UQz9qsNw-

1777239662-1.0.1.1-FwvGNesiXbh9vbtKGwFanmHOdTXK.hBnlwem38UY2p8 only listing of second hand literature

Link chosen: https://muj-antikvariat.cz/kniha/studie-z-poetiky-mukarovsky-jan-1982?__cf_chl_tk=bVS0FGMl51Y6DzdOW9fDcNCl_dH4DEm5wq.UQz9qsNw-1777239662-1.0.1.1-FwvGNesiXbh9vbtKGwFanmHOdTXK.hBnlwem38UY2p8

Search Engine: Google Scholar

Date and Time: 07/05/2026 15:46

Search query: Mukařovský (1982)

Link visited with pros and cons for each link visited: did not visit any links, because they provided only references to Mukařovksý's article, rather than the article itself

Link chosen: none

Search Engine: Google

Date and Time: 07/05/2026 15:46

Search query: babička txt

Link visited with pros and cons for each link visited:
https://is.muni.cz/th/174261/ff_b/Babicka.txt contains a txt file of a senior thesis rather than of the book itself.

https://web2.mlp.cz/koweb/00/03/34/76/44/babicka.pdf very helpful, contains pdf of the entire book Babička with the text being encoded as text and not as an image.

Link chosen: https://web2.mlp.cz/koweb/00/03/34/76/44/babicka.pdf

---

Search Engine: Google

Date and Time: 07/05/2026 15:46

Search query: slova se slabším přízvukem na konci vět babička

Link visited with pros and cons for each link visited:
https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://gummidek.estranky.cz/clanky/mukarovsky---pokus-o-slohovy-rozbor-babicky-bozeny-nemcove.html&ved=2ahUKEwj92eCCxYyUAxUrXmwGHQOcMD4QFnoECB8QAQ&usg=AOvVaw1M3e1WcaCXcugj5ewwuBKh very helpful, provided some information about how the sentences end, including how Němcová swaps words in the end of the sentence to ensure that the sentence ends with the softer one.

Link chosen:
https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://gummidek.estranky.cz/clanky/mukarovsky---pokus-o-slohovy-rozbor-babicky-bozeny-nemcove.html&ved=2ahUKEwj92eCCxYyUAxUrXmwGHQOcMD4QFnoECB8QAQ&usg=AOvVaw1M3e1WcaCXcugj5ewwuBKh

---

Search Engine: Google

Date and Time: 07/05/2026 15:46

Search query: knihy data txt

Link visited with pros and cons for each link visited:

https://www.google.com/url?sa=i&source=web&rct=j&url=https://pdfknihy.maxzone.eu/&ved=2ahUKEwiBt-uq2pKUAxW9r1YBHcGnOlgQy_kOegQIARAC&opi=89978449&cd&psig=AOvVaw0IyX5Hr2MPgXE1JIdT038M&ust=1777539914808000 – there are some books, but not too many options for a full representative dataset

https://www.kaggle.com/datasets/durgaprasad64/sample-text-data-set - not very helpful, since it does not contain Czech text

https://www.reddit.com/r/DataHoarder/comments/fyb6gt/4_million_txt_books_in_537gb/?tl=cs – potentially helpful, someone recommends library genesis

Link chosen:
https://www.google.com/url?sa=i&source=web&rct=j&url=https://pdfknihy.maxzone.eu/&ved=2ahUKEwiBt-

uq2pKUAxW9r1YBHcGnOlgQy_kOegQIARAC&opi=89978449&cd&psig=AOvVaw0IyX5Hr2MPgXE1JIdT038M&ust=1777539914808000

Search Engine: Google

Date and Time: 07/05/2026 15:46

Search query: library genesis

Link visited with pros and cons for each link visited:
https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://libgen.im/&ved=2ahUKEwjS5qae3JKUAxVkrq8BHS0U2EQFnoECAwQAQ&usg=AOvVaw1Vja0fqJjtHihUwI9XEGgz – complicated, does not allow for selective download of Czech books.

Link chosen: none

Search Engine: Google

Date and Time: 07/05/2026 15:46

Search query: městská knihovna v praze zdarma pdf

Link visited with pros and cons for each link visited:
https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://libgen.im/&ved=2ahUKEwjS5qae3JKUAxVkrq8BHS0U2EQFnoECAwQAQ&usg=AOvVaw1Vja0fqJjtHihUwI9XEGgz – complicated, does not allow for selective download of Czech books.

Link chosen: none

Search Engine: Google

Date and Time: 07/05/2026 15:46

Search query: wikipedie stáhnout

Link visited with pros and cons for each link visited:
https://cs.wikipedia.org/wiki/Wikipedie:St%C3%A1hnut%C3%AD_datab%C3%A1ze – very helpful, contains a multitude of formats for downloading the Czech Wikipedia.

Link chosen:
https://cs.wikipedia.org/wiki/Wikipedie:St%C3%A1hnut%C3%AD_datab%C3%A1ze

---

Search Engine: Google

Date and Time: 07/05/2026 15:46

Search query: dělení souhlásek podle způsobu použití svalů k výslovnosti

Link visited with pros and cons for each link visited:
https://www.google.com/url?sa=t&source=web&rct=j&url=https%3A%2F%2Fis.muni.cz%2Fel%2F1441%2Fpodzim2017%2FCJp005%2Fum%2Ffonetika_a_fonologie_klasifikace_hlasek_II_souhlasky.pdf&ved=0CAEQ1fkOahcKEwiI_4XO2ZSUAxUAAAAAHQAAAAAQAw&opi=89978449 – very helpful resource on how consonants are phonetically divided in Czech.

Link chosen:
https://www.google.com/url?sa=t&source=web&rct=j&url=https%3A%2F%2Fis.muni.cz%2Fel%2F1441%2Fpodzim2017%2FCJp005%2Fum%2Ffonetika_a_fonologie_klasifikace_hlasek_II_souhlasky.pdf&ved=0CAEQ1fkOahcKEwiI_4XO2ZSUAxUAAAAAHQAAAAAQAw&opi=89978449

Search Engine: Google

Date and Time: 07/05/2026 15:46

Search query: babička

Link visited with pros and cons for each link visited: no helpful links, all in english

Link chosen:

Search Engine: Google

Date and Time: 07/05/2026 15:46

Search query: babička česká wikipedie

Link visited with pros and cons for each link visited:
https://cs.wikipedia.org/wiki/Babi%C4%8Dka_(kniha) – helpful, contains references

Link chosen: https://cs.wikipedia.org/wiki/Babi%C4%8Dka_(kniha)

Search Engine: Google

Date and Time: 07/05/2026 15:46

Search query: how to extract all the wikipedia articles from the downlaoded dataset

Link visited with pros and cons for each link visited:
https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://medium.com/%40hps257/ways-of-extracting-data-from-wikipedia-23ffd77f6902&ved=2ahUKEwiG49K-m5eUAxXnsVYBHYxNHQ4QFnoECDQQAQ&usg=AOvVaw1NDzdRfpi0aoPtiRYSrvqr – the article was helpful, and provided some scripts that I could use.

Link chosen:
https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://medium.com/%40hps257/ways-of-extracting-data-from-wikipedia-23ffd77f6902&ved=2ahUKEwiG49K-m5eUAxXnsVYBHYxNHQ4QFnoECDQQAQ&usg=AOvVaw1NDzdRfpi0aoPtiRYSrvqr

---

Search Engine: Google

Date and Time: 07/05/2026 15:51

Search query: how to ensure that date and time do not update word

Link visited with pros and cons for each link visited: AI summary was helpful to use the shortcut

Link chosen: none

Search Engine: Google

Date and Time: 07/05/2026 15:54

Search query: how to connect google colab to github

Link visited with pros and cons for each link visited:
https://github.com/WiktorJ/google-colab-tutorial - helpful for storing the notebook in github

Link chosen: https://github.com/WiktorJ/google-colab-tutorial

---

Search Engine: Google

Date and Time: 07/05/2026 15:57

Search query: how to load files from github into google colab

Link visited with pros and cons for each link visited: AI summary was helpful in pointing out that GitHub can make files accessible in raw format through URL.

Link chosen: none

Search Engine: Google

Date and Time: 07/05/2026 20:24

Search query: how to load files from github into google colab

Link visited with pros and cons for each link visited:
https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://github.com/orgs/community/discussions/22534&ved=2ahUKEwjr3rf836aUAxW7sVYBHZorDSgQFnoECBkQAQ&usg=AOvVaw3mlLBHV-O2ncWOei48ulpU – useful, provides tutorial

Link chosen:
https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://github.com/orgs/community/discussions/22534&ved=2ahUKEwjr3rf836aUAxW7sVYBHZorDSgQFnoECBkQAQ&usg=AOvVaw3mlLBHV-O2ncWOei48ulpU

Search Engine: Google

Date and Time: 07/05/2026 20:40

Search query: load data from google drive to colab¨

Link visited with pros and cons for each link visited:
https://colab.research.google.com/notebooks/io.ipynb#scrollTo=RWSJpsyKqHjH– very helpful, provides sample code

Link chosen:
https://colab.research.google.com/notebooks/io.ipynb#scrollTo=RWSJpsyKqHjH

Search Engine: Google

Date and Time: 07/05/2026 21:04

Search query: pypdfium2

Link visited with pros and cons for each link visited:
https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://pypi.org/project/pypdfium2/&ved=2ahUKEwjvjOjz6KaUAxVfoa8BHQ5RJGEQFnoECCAQAQ&usg=AOvVaw1LeWSSqGSKn7T8PAPqoWrE – helpful, provides all info for pypdfium2

Link chosen:
https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://pypi.org/project/pypdfium2/&ved=2ahUKEwjvjOjz6KaUAxVfoa8BHQ5RJGEQFnoECCAQAQ&usg=AOvVaw1LeWSSqGSKn7T8PAPqoWrE

Search Engine: Google

Date and Time: 07/05/2026 21:20

Search query: dask bag load files

Link visited with pros and cons for each link visited:
https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://docs.dask.org/en/latest/bag-creation.html&ved=2ahUKEwjLqf-w66aUAxWAi68BHTt8JHMQFnoECB0QAQ&usg=AOvVaw3G5tuSxIKdpZ2qcTiSHttt – very helpful, provides examples

Link chosen:
https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://docs.dask.org/en/latest/bag-creation.html&ved=2ahUKEwjLqf-w66aUAxWAi68BHTt8JHMQFnoECB0QAQ&usg=AOvVaw3G5tuSxIKdpZ2qcTiSHttt

---

Search Engine: Google

Date and Time: 09/05/2026 19:51

Search query: how to load all files in a folder to google colab

Link visited with pros and cons for each link visited: no links were opened because they did not look helpful

Link chosen: none

Search Engine: Google

Date and Time: 09/05/2026 23:09

Search query: \r meaning

Link visited with pros and cons for each link visited: AI summary helpful

Link chosen: none

Search Engine: Google

Date and Time: 09/05/2026 23:11

Search query: python string processing

Link visited with pros and cons for each link visited: https://www.w3schools.com/python/python_ref_string.asp very helfpul, describes various methods

Link chosen: https://www.w3schools.com/python/python_ref_string.asp

---

Search Engine: Google

Date and Time: 10/05/2026 08:54

Search query: python string ascii

Link visited with pros and cons for each link visited: https://www.google.com/url?sa=i&source=web&rct=j&url=https://docs.python.org/3/lib

rary/functions.html&ved=2ahUKEwip7vSPi62UAxX6sVYBHQiAJPwQy_kOegQIAxAC&opi=89978449&cd&psig=AOvVaw1u-gVL47iflz7sQRMoauNS&ust=1778446364874000 very helfpul, describes various methods

Link chosen:
https://www.google.com/url?sa=i&source=web&rct=j&url=https://docs.python.org/3/library/functions.html&ved=2ahUKEwip7vSPi62UAxX6sVYBHQiAJPwQy_kOegQIAxAC&opi=89978449&cd&psig=AOvVaw1u-gVL47iflz7sQRMoauNS&ust=1778446364874000

Search Engine: Google

Date and Time: 10/05/2026 13:49

Search query: seznam ceskych zkratek atd.

Link visited with pros and cons for each link visited: https://cja.ujc.cas.cz/e-cja/zkratky - very useful, contains a list of the most frequent abbreviations.

Link chosen: https://cja.ujc.cas.cz/e-cja/zkratky

Search Engine: Google

Date and Time: 10/05/2026 14:27

Search query: regex python

Link visited with pros and cons for each link visited:
https://www.w3schools.com/python/python_ref_string.asp - very helpful, provides a lot of documentation and examples

Link chosen: https://www.w3schools.com/python/python_ref_string.asp

---

Search Engine: Google

Date and Time: 10/05/2026 14:27

Search query: python regex lookahead

Link visited with pros and cons for each link visited:
https://www.geeksforgeeks.org/python/python-regex-lookahead/ - very helpful, provides examples.

Link chosen: https://www.geeksforgeeks.org/python/python-regex-lookahead/

Search Engine: Google

Date and Time: 17/05/2026 19:35

Search query: pandas dataframe

Link visited with pros and cons for each link visited:
https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html - very helpful documentation

Link chosen: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html

---

Search Engine: Google

Date and Time: 20/05/2026 21:37

Search query: which regular expression are computationally expensive

Link visited with pros and cons for each link visited: AI summary helpful outlining the basics

Link chosen: none

Search Engine: Google

Date and Time: 20/05/2026 23:32

Search query: všechny předložky

Link visited with pros and cons for each link visited:
https://cs.wiktionary.org/wiki/Kategorie:%C4%8Cesk%C3%A9_p%C5%99edlo%C5%BEky – very helpful, contains list of all the prepositions

Link chosen:
https://cs.wiktionary.org/wiki/Kategorie:%C4%8Cesk%C3%A9_p%C5%99edlo%C5%BEky

---

Search Engine: Google

Date and Time: 21/05/2026 23:14

Search query: how to join list of words together into string python

Link visited with pros and cons for each link visited:
https://www.w3schools.com/python/ref_string_join.asp did not have results of examples, but the example did what I needed, so good

Link chosen: https://www.w3schools.com/python/ref_string_join.asp

---

Search Engine: Google

Date and Time: 23/05/2026 23:16

Search query: how to exclude column in dataframe

Link visited with pros and cons for each link visited: AI summary helpful

Link chosen: none

Search Engine: Google

Date and Time: 24/05/2026 19:50

Search query: how to measure time in python

Link visited with pros and cons for each link visited: AI summary helpful

Link chosen: none

Search Engine: Google

Date and Time: 24/05/2026 21:05

Search query: cannot see dask distributed dashboard when running in google colab

Link visited with pros and cons for each link visited: AI summary helpful

Link chosen: none

Search Engine: Github Copilot

Date and Time: 25/05/2026 20:48

Search query: why is the flatten operation being so expensive when converting used on a dask bag of lists of sentences. I would like to analyze and parallelyze the text at sentence level, but whenever I try to do that, the performance is severely hindered
Output: Optimized tool selection

Starting: *Suggest code improvements for efficiency* (3/4)

`flatten()` is expensive in your pipeline because it's the moment where you turn "a few large items" into "millions of tiny Python objects", and Dask Bag (and Python) pay a lot of overhead per element.

In code(1).ipynb.ipynb) you effectively do:

- `to_sentences()` returns a **Python list** of sentence strings (allocates lots of objects at once).

- `.flatten()` then **concatenates/iterates those lists** to create a new bag whose *logical* element count explodes.

- Every later `.map/.filter` now runs **per sentence**, so you're doing millions of Python calls + regex/split work + (with `distributed`) serialization between worker tasks.

Why it hurts so much

- **Python object overhead dominates**: a 1–2 MB text chunk can become tens of thousands of sentence strings; the bytes aren't the problem, the *object count* is.

- **Serialization/network overhead (distributed)**: millions of small strings moving between tasks/workers is far slower than keeping work inside one task and emitting only aggregated results.

- **Low partition parallelism makes it worse**: with `files_per_partition=50` you create very few partitions, so each task produces a gigantic list-of-sentences, then `flatten` has to churn through it.

Best fix: avoid `flatten()` by emitting sentences directly at partition level

Instead of "map to list, then flatten", do one `map_partitions` that *yields* sentences:

```python
import re

split_re = re.compile(r"[.!?:]")

def partition_to_sentences(partition):
    for name, text in partition:
        for s in split_re.split(text):
            s = s.strip()
            if s:
                yield (name, s)

sentences = all_text.map_partitions(partition_to_sentences)
```

Even better: push your cleaning + filters into that same generator so each sentence is processed once (no extra passes).

If you truly need sentence-level data

- Keep partitions reasonably sized (e.g., `files_per_partition=1..5`, not 50), so sentence work is spread across cores.

- Avoid calling `.compute()` on the full sentence bag; prefer computing **aggregations** (counts, frequencies, summary stats) or `persist()` and continue downstream.

If you tell me roughly how many sentences you end up with (e.g., `sentences.count().compute()`), I can recommend a good target for `files_per_partition` / `npartitions` so you get parallelism without drowning in per-sentence overhead.

Completed: *Suggest code improvements for efficiency* (3/4)

Search Engine: Google

Date and Time: 26/05/2026 13:11

Search query: flatten multilevel dataframe

Link visited with pros and cons for each link visited: https://github.com/pandas-dev/pandas/issues/19950 - very helpful, provided a function to flatten the multilevel dataframe

Link chosen: https://github.com/pandas-dev/pandas/issues/19950

Search Engine: Google

Date and Time: 26/05/2026 15:27

Search query: t test python

Link visited with pros and cons for each link visited:
https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ttest_ind.html - very helpful, provides documentation of the scipy library and how I can use it to perform a t-test

Link chosen:
https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ttest_ind.html

---

Search Engine: Google

Date and Time: 26/05/2026 23:27

Search query: series to dataframe

Link visited with pros and cons for each link visited:
https://pandas.pydata.org/docs/reference/api/pandas.Series.to_frame.html - very useful, provides documentation

Link chosen:
https://pandas.pydata.org/docs/reference/api/pandas.Series.to_frame.html

---

Search Engine: Google

Date and Time: 27/05/2026 09:00

Search query: calculate t test of two t tests

Link visited with pros and cons for each link visited: https://imaging.mrc-cbu.cam.ac.uk/statswiki/FAQ/td - very helpful, provides various resources

Link chosen: https://imaging.mrc-cbu.cam.ac.uk/statswiki/FAQ/td

---

Search Engine: Google

Date and Time: 29/05/2026 12:30

Search query: link to sections of a google colab in google colab

Link visited with pros and cons for each link visited: AI summary helpful

Link chosen: none

Search Engine: Google

Date and Time: 29/05/2026 12:30

Search query: how to include installations in a python notebook that writes a file to be interpreted by a python interpreter

Link visited with pros and cons for each link visited: AI summary helpful

Link chosen: none

Search Engine: Google

Date and Time: 29/05/2026 12:30

Search query: google dataproc cluster for dask

Link visited with pros and cons for each link visited:
https://medium.com/@abhilashagulhane111/scaling-data-processing-with-dask-on-google-cloud-bb2c41843bab - extremely helpful, provides a very simple implementation of parallel processing on a distributed dask cluster

Link chosen: https://medium.com/@abhilashagulhane111/scaling-data-processing-with-dask-on-google-cloud-bb2c41843bab

---